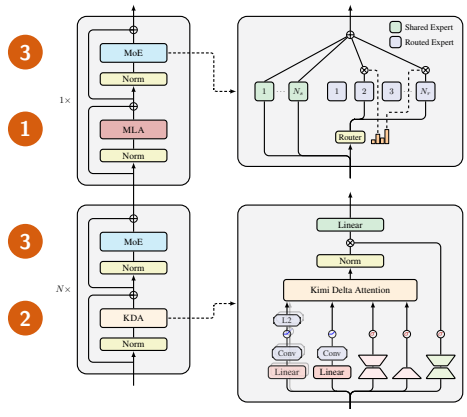
Kimi Linear

An Expressive, Efficient Attention Architecture

Kimi Team, November 2025 · arXiv:2510.26692

Presenter: Zhihan Yang

Agenda



1 Making Attention Cheaper (Still Quadratic) 2 Making Linear Attention Work 3 Making the MLP Cheaper

Kimi Linear, Figure 3 — 3 KDA layers per MLA layer, each followed by MoE.

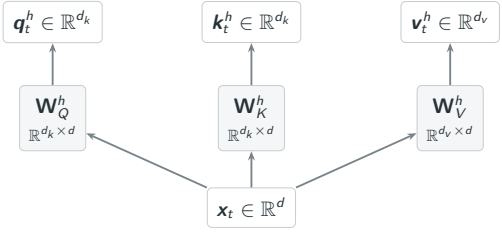
Section 1

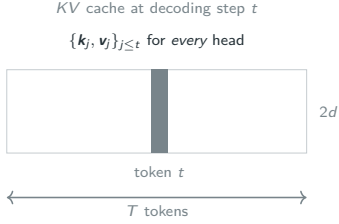
Making Attention Cheaper (Still Quadratic)

Multi-head self-attention (MHA)

$$\boldsymbol{o}_t = \mathbf{W}_o \,\mathrm{Concat}_{h=1}^{H}\left(\boldsymbol{o}_t^h\right) \in \mathbb{R}^d$$

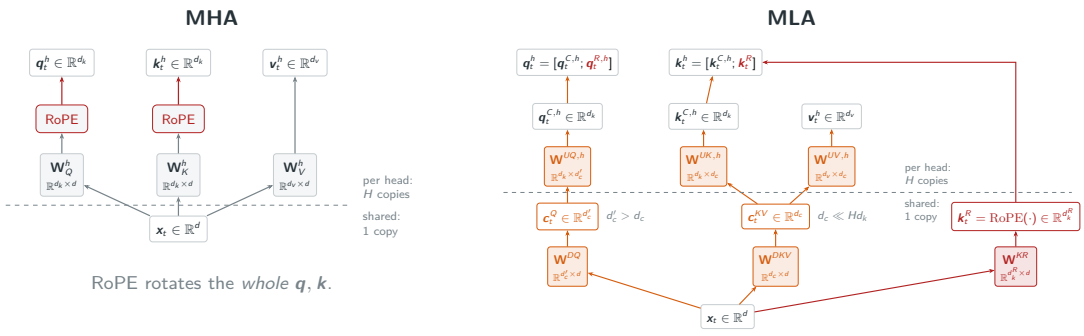
$$\boldsymbol{o}_t^h = \sum_{j \le t} \mathrm{softmax}_{j \le t}\left(\frac{\left(\boldsymbol{q}_t^h\right)^\top \boldsymbol{k}_j^h}{\sqrt{d_k}}\right) \boldsymbol{v}_j^h \in \mathbb{R}^{d_v}$$





- Cache cost grows **linearly in** T.
- Per-step compute grows linearly too.
- $d_k = d_v = d/H$.

Multi-head latent attention (MLA) — Training



RoPE rotates the *whole* $\boldsymbol{q}$, $\boldsymbol{k}$.

low-rank compression decoupled RoPE D: down · U: up · R: RoPE

But now position enters *only* through the d_k^R dimensions ($\boldsymbol{q}_t^{R,h}$ is RoPE of $\boldsymbol{c}_t^Q$)

Intuition: MLA is MHA with every projection factorized through a low-rank latent ($\boldsymbol{c}_t^Q$, $\boldsymbol{c}_t^{KV}$)

$$\mathbf{W}_Q^h \approx \mathbf{W}^{UQ,h}\mathbf{W}^{DQ} \qquad \mathbf{W}_K^h \approx \mathbf{W}^{UK,h}\mathbf{W}^{DKV} \qquad \mathbf{W}_V^h \approx \mathbf{W}^{UV,h}\mathbf{W}^{DKV}$$

MLA vs MHA: measured quality and cache

	Small MoE (1.33T tokens)		Large MoE (420B tokens)	
	MHA	MLA	MHA	MLA
Activated params	2.5B	2.4B	25.0B	21.5B
KV cache / token	110.6K	15.6K	860.2K	34.6K
BBH	37.9	39.0	46.6	50.7
MMLU	48.7	50.0	57.5	59.0
C-Eval	51.6	50.9	57.9	59.2
CMMLU	52.3	53.4	60.7	62.5

DeepSeek-V2, Appendix D (Tables 8–9). Attention swapped in place — same depth, width, heads, data; params land where they land, so MLA is handicapped.

- MLA wins 7 of 8, with *fewer* activated params; cache falls to **14%** / **4%** of MHA (independent of T).
- Margin widens with scale ($+0.7 \rightarrow +2.4$) — but two points is not a scaling law.
- 7B dense: MHA $>$ GQA $>$ MQA on MMLU (45.2/41.2/37.9) — cheaper caches usually cost quality.
- Caveat: only these four (three multiple-choice, all short-context) — where a compressed cache is *not* stressed. GSM8K/MATH/HumanEval appear only for the shipped model, no MHA baseline.

Multi-head latent attention (MLA) — Inference

Past tokens are never expanded back into $\boldsymbol{k}, \boldsymbol{v}$ — attention runs on the latent.

Scores: the two halves are handled differently.

$$\boldsymbol{q}_t^{h\top}\boldsymbol{k}_j^h = \Big(\underbrace{\boldsymbol{q}_t^{C,h\top}\mathbf{W}^{UK,h}}_{\text{fold: once per step}}\Big)\boldsymbol{c}_j^{KV} + \underbrace{\boldsymbol{q}_t^{R,h\top}\boldsymbol{k}_j^R}_{\text{vs. cached } \boldsymbol{k}_j^R}$$

Values: pull $\mathbf{W}^{UV}$ out of the sum.

$$\sum_{j\le t} a_j\left(\mathbf{W}^{UV,h}\boldsymbol{c}_j^{KV}\right) = \mathbf{W}^{UV,h}\left(\sum_{j\le t} a_j\,\boldsymbol{c}_j^{KV}\right)$$

- The fold depends on t, not j: done **once** per step.
- Only the *content* half is absorbed; under **NoPE** the second term vanishes.
- $\boldsymbol{k}_j^{C,h}, \boldsymbol{v}_j^h$ are never stored *and never rebuilt*.
- $\boldsymbol{c}_t^Q$ is **not** cached — queries are current-token only.



DeepSeek-V3 config; traffic per token, per layer

- This is exactly **MQA** with head dim d_c.
- Decoding is bandwidth-bound: $\sim$57$\times$ less traffic per step.

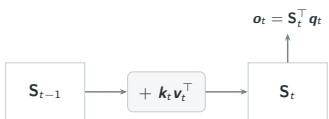
Section 2

Making Linear Attention Work

Linear attention: a fixed-size state

Drop the softmax. Keep a matrix-valued state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ instead of a growing cache.

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{k}_t \boldsymbol{v}_t^\top, \qquad \boldsymbol{o}_t = \mathbf{S}_t^\top \boldsymbol{q}_t$$



size of $\mathbf{S}_t$ is **independent of** T

Online-learning view. The update is gradient *descent* on

$$\mathcal{L}_t(\mathbf{S}) = -\langle \mathbf{S}^\top \boldsymbol{k}_t,\ \boldsymbol{v}_t \rangle$$



all writes accumulate

- $\mathbf{S}_t$ is an *associative memory*: key $\mapsto$ value.
- $O(1)$ memory, $O(1)$ compute per token.

- No criterion for *what to erase*.
- Associations interfere over long contexts.

Fixed memory, but no forgetting and no error correction.

DeltaNet: error-correcting updates

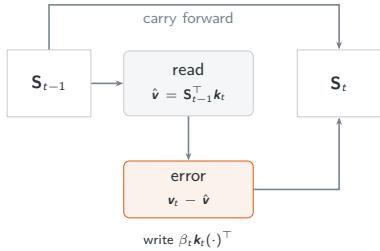
Replace the correlation objective with a reconstruction loss: $\mathcal{L}_t(\mathbf{S}) = \frac{1}{2}\|\mathbf{S}^\top \boldsymbol{k}_t - \boldsymbol{v}_t\|^2$.

One gradient step with learning rate β_t:

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \beta_t \nabla_{\mathbf{S}} \mathcal{L}_t(\mathbf{S}_{t-1}) = \left(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top\right)\mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top$$

Equivalently, as an *error-correcting* write:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \beta_t\, \boldsymbol{k}_t \Big(\underbrace{\boldsymbol{v}_t - \mathbf{S}_{t-1}^\top \boldsymbol{k}_t}_{\text{prediction error}} \Big)^\top$$



- $\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top$ is a generalized **Householder** transform: erase along $\boldsymbol{k}_t$ only.
- Rank-1 structure $\Rightarrow$ chunkwise parallel training.

If $\mathbf{S}_{t-1}$ already predicts $\boldsymbol{v}_t$, nothing is written.

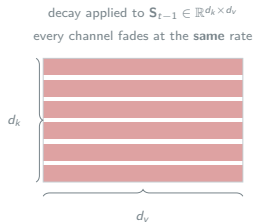
Error correction, but old associations persist indefinitely.

Gated DeltaNet: scalar decay

Add a data-dependent forget gate $\alpha_t \in [0, 1]$ — a scalar per token.

$$\mathbf{S}_t = \alpha_t \left(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top\right) \mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top$$





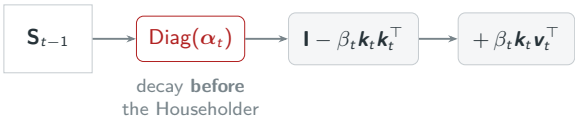
- α_t acts as **weight decay** on the fast weights — data-dependent L_2 regularization.
- Bounds memory lifespan; mitigates interference.
- Also a *multiplicative positional encoding*: a learnable, data-dependent transition matrix.

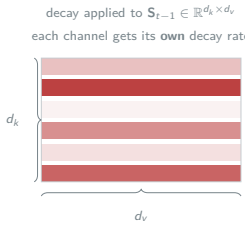
A single gate cannot keep one channel while flushing another.

Kimi Delta Attention (KDA): fine-grained gating

Promote the scalar α_t to a per-channel vector $\boldsymbol{\alpha}_t \in [0,1]^{d_k}$.

$$\mathbf{S}_t = \left(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top\right) \mathrm{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top$$
$$\boldsymbol{o}_t = \mathbf{S}_t^\top \boldsymbol{q}_t \in \mathbb{R}^{d_v}$$





Parameterization (low-rank, like the output gate):

$$\boldsymbol{\alpha}_t^h = f\left(\mathbf{W}_\alpha^\uparrow \mathbf{W}_\alpha^\downarrow \boldsymbol{x}_t\right) \in [0,1]^{d_k}, \quad \beta_t^h = \sigma(\mathbf{W}_\beta^h \boldsymbol{x}_t)$$

- Per-channel forgetting *and* positional awareness.
- DPLR expressiveness, but tying $\boldsymbol{a}_t, \boldsymbol{b}_t$ to $\boldsymbol{k}_t$ makes the kernel $\sim 2\times$ faster.

Summary of state updates

Model	State update	Decay	Memory
Linear attn.	$\mathbf{S}_{t-1} + \boldsymbol{k}_t \boldsymbol{v}_t^\top$	none	$d_k \times d_v$
DeltaNet	$(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top)\mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top$	none	$d_k \times d_v$
Gated DeltaNet	$\alpha_t(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top)\mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top$	scalar $\alpha_t \in [0,1]$	$d_k \times d_v$
KDA	$(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\top)\mathrm{Diag}(\boldsymbol{\alpha}_t)\mathbf{S}_{t-1} + \beta_t \boldsymbol{k}_t \boldsymbol{v}_t^\top$	$\boldsymbol{\alpha}_t \in [0,1]^{d_k}$ per channel	$d_k \times d_v$

State size is $O(1)$ in every row; the variants differ only in the transition matrix.

Section 3

Making the MLP Cheaper